System Design Document

Personal Project

C. Stijlaart

Table of Contents

1. Document history

Table 1. Document History

Version	Date	Status	Author	Description	Remarks
0.1	10-10-2023	Draft	Casey Stijlaart	Introduction, Description Component diagram	-
0.2	11-11-2023	Draft	Casey Stijlaart	Completed design	-
0.3	16-11-2023	Dradt	Casey Stijlaart	Completed system behavior	-

2. Introduction

This document serves as a comprehensive guide detailing the design and architecture of the embedded system that transforms environmental data into musical compositions. It contains a detailed system description with multiple diagrams defining all of the hardware components and the behavior of the system.

3. System Description

The system is designed to transform environmental data into musical compositions through the integration of of various components. The system aims to provide users with an intuitive and engaging experience, offering real-time insights into their environment in the form of unique musical compositions. As the musical composition has to trigger an emotion based on the current environmental data, therefor, when a value from a sensor is consider poor, the musical composition should be more dark and sinister. When the environmental data is considered good, the musical composition should be more bright and calming. Each musical composition will be based upon a different data processing approach.

3.1. Extensions

Should

- Generate a musical composition based upon environmental data.
- Collect environmental data.
- Change the data processing approach during run time.
- Store any collected environmental data in a database.

Could

- Allow for addition of extra sensors.

4. System Design

4.1. Components

To create the system, different components will work together. To outline each component and their role, in Table 2 an overview is given. In Figure 1 the relation between the different components are outlined.

Table 2. Component Description

Component	Description
Controller Unit (CU)	The CU serves as the central processing unit for the system.
Environmental Data Unit (EDU)	The EDU collects data from the system`s environment.
Database (DB)	The DB stores and manages data generated by the system.
Interface (IF)	The IF component provides an application interface for external interactions, trough an user, with the system.

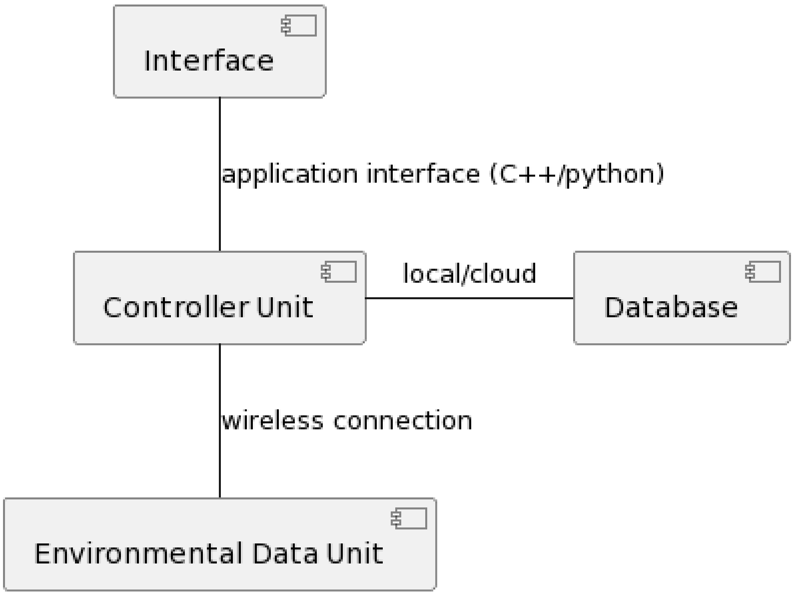


Figure 1. Component Diagram

To showcase how each component is connected to another, the overall message that connects them are defined, see [Messages between components].

Table 3. Messages between components

Component	Message name	From	To	Description
Interface	DisplayAlgorithm(`algorithm`)	Controller Unit	Interface	Contains the used algorithm (`algorithm`).

Component	Message name	From	To	Description
Controller Unit	ChangeAlgorithm(`algorithm`)	Interface	Controller Unit	Allows for change of the used algorithm for the musical composition.
Database	StoreData(`sensor_id`, `data`)	Controller Unit	Database	Stores the sensor data in the database
Database	RetrieveStoredData(`sensor_id`,`data`)	Controller Unit	Database	Contains the data (`data`) for the sensor (`sensor`)
Environmental Data Unit	RetrieveSensorData(`sensor_id`, `sensor_data`)	Controller Unit	Environmental Data Unit	Retrieves the measured sensor data for the given sensor ID (`sensor_id`).

4.1.1. Use Cases

In this section use cases and corresponding requirements are described.

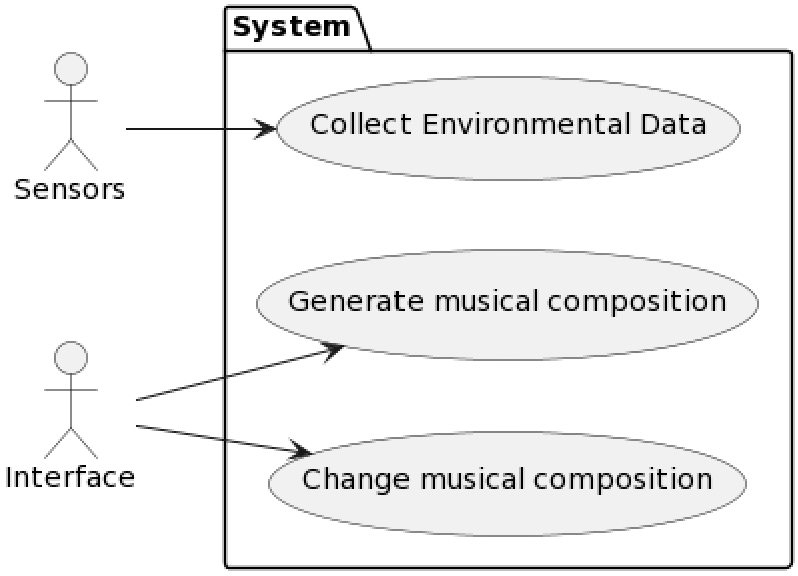


Figure 2. Use Case Diagram

[Use Case Diagram] depicts some of the use cases of {project-name}. It shows system boundary and specifies how actors interact with the system. Each use case will be described in the following table.

Table 4. Use Case 01

ID: UC01	Use Case Name: Collect Environmental Data
Description	When the system is running, environmental data is collected and stored.

ID: UC01	**Use Case Name: Collect Environmental Data**
Actor(s)	Environmental Data Unit (EDU), Database (DB)

Table 5. Use Case 02

ID: UC02	**Use Case Name: Generate musical composition**
Description	When the user request for a musical composition to be generated, the controller unit uses the environmental data to generate the musical composition.
Actor(s)	Control Unit (CU), Interface (IF), Database(DB)

Table 6. Use Case 03

ID: UC03	**Use Case Name: Change musical composition**
Description	When the user changes the used algorithm trough the interface, the musical composition output changes.
Actor(s)	Interface (IF)

4.2. Requirements

A system has various requirements, each one covering an important aspect of how the system must work or behavior. These requirements are divided into the user-, functional- and non-functional requirements.

4.2.1. User Requirements

Table 7. User Requirements

ID	Requirement	Use Case(s)
UR01	The system should be able to start collecting environmental data at startup.	UC01
UR02	Users should be able to adjust the musical composition output settings through the interface.	UC02/UC03
UR03	The system should provide a musical compositions, which triggers emotions, based on the collected environmental data.	UC02

4.2.2. Functional Requirements

Table 8. Functional Requirements

ID	Requirement	Use Case(s)
FR01	The EDU must be able to collect data.	UC01
FR02	The CU must be able to request measured environmental data from the EDU and DB to process it using algorithms.	UC01/UC02
FR03	The CU must be able to interfaces with the DB for storing and retrieving environmental data.	UC01
FR04	The IF and CU must be able to adjust the current used musical output settings.	UC02/UC03
FR05	The IF must be able to showcase the current system status	UC03

4.2.3. Non-Functional Requirements

Table 9. Non-Functional Requirements

ID	Requirement
NFR01	The system should be able to handle environmental data from multiple sensors simultaneously.
NFR02	The musical compositions should be designed to evoke specific human emotions effectively.
NFR03	The system should be scalable to accommodate future upgrades or additions to environmental sensors.

4.3. Requirements traceability matrix

To ensure all use cases are covered with the set requirements, a traceability matrix is created. The coverage is showcased in [Use cases vs user requirements coverage] and [Functional requirements vs user requirements coverage]

Table 10. Use cases vs user requirements coverage

	UC01	**UC02**	**UC03**
UR01	X		
UR02		X	X
UR03		X	

Table 11. Functional requirements vs user requirements coverage

	UC01	**UC02**	**UC03**
FR01	X		
FR02	X	X	
FR03	X		
FR04		X	X
FR05			X

5. System behavior

5.1. Sequence diagrams

In this section different scenarios are considered per use case and the corresponding interactions are described by sequence diagrams. The modules from the previous section are used as objects in these diagrams. In some sequence diagrams the actors from the use cases appear to be an object in a diagram, since they Interact with the system by sending or receiving messages.

5.1.1. UC_01 : Collect Environmental Data

Scenario 1

When the user initiates the start of collecting, the sensor data is captured and stored in the database.

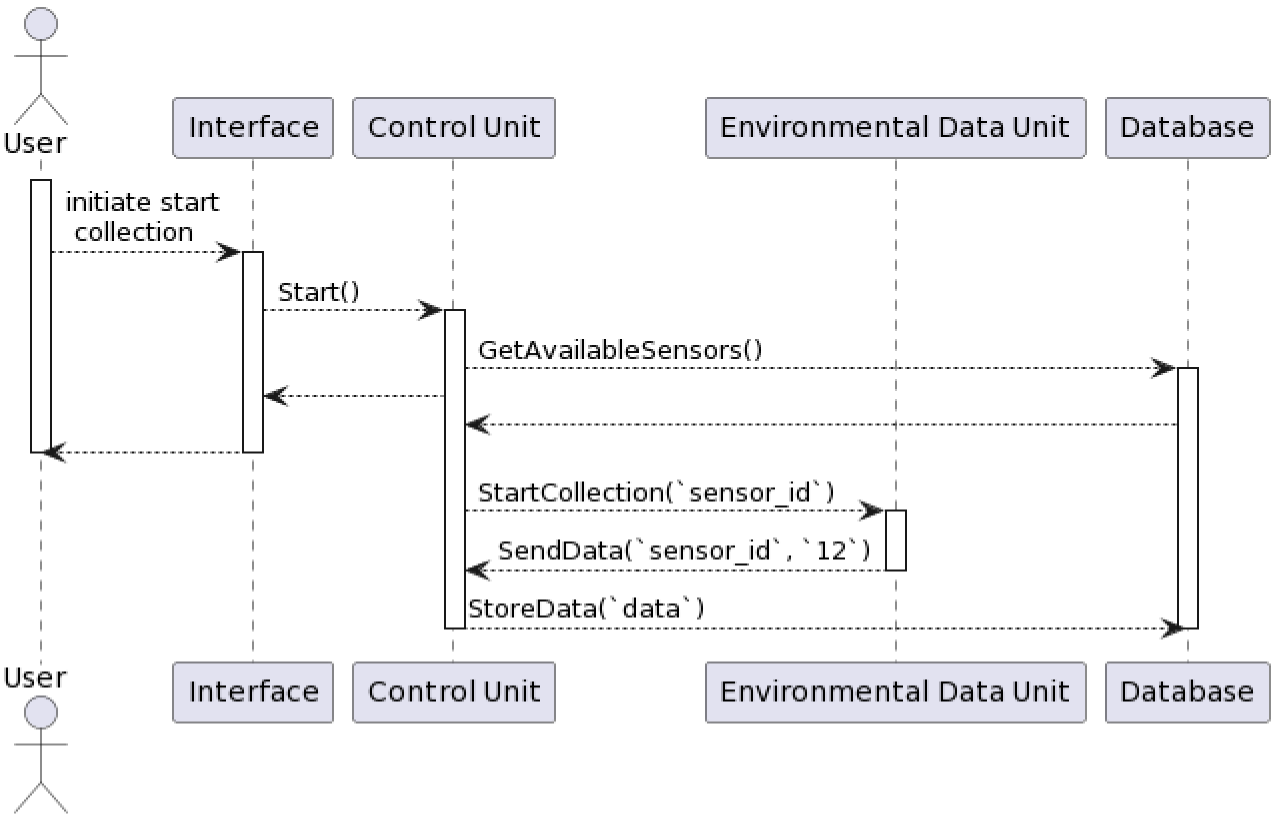


Figure 3. UC_01 Scenario 1

Scenario 2

When the user tries to initiate the start of collecting data while no sensors are connected, an error message occurs.

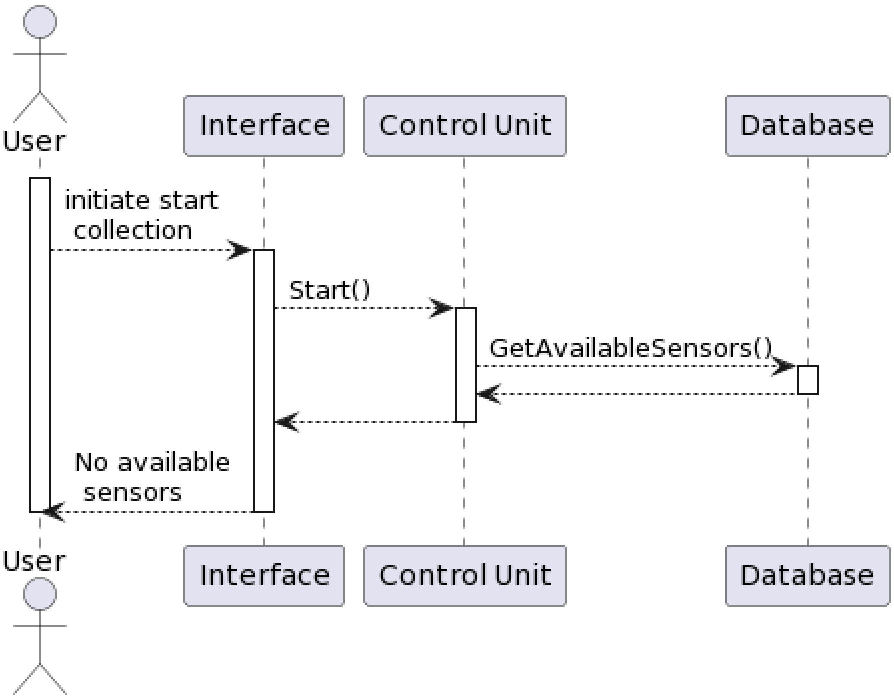


Figure 4. UC_01 Scenario 2

5.1.2. UC_02 : Generate musical composition

Scenario 1

When the user initiates the start of starting the musical composition, the musical composition is genereted and outputted.

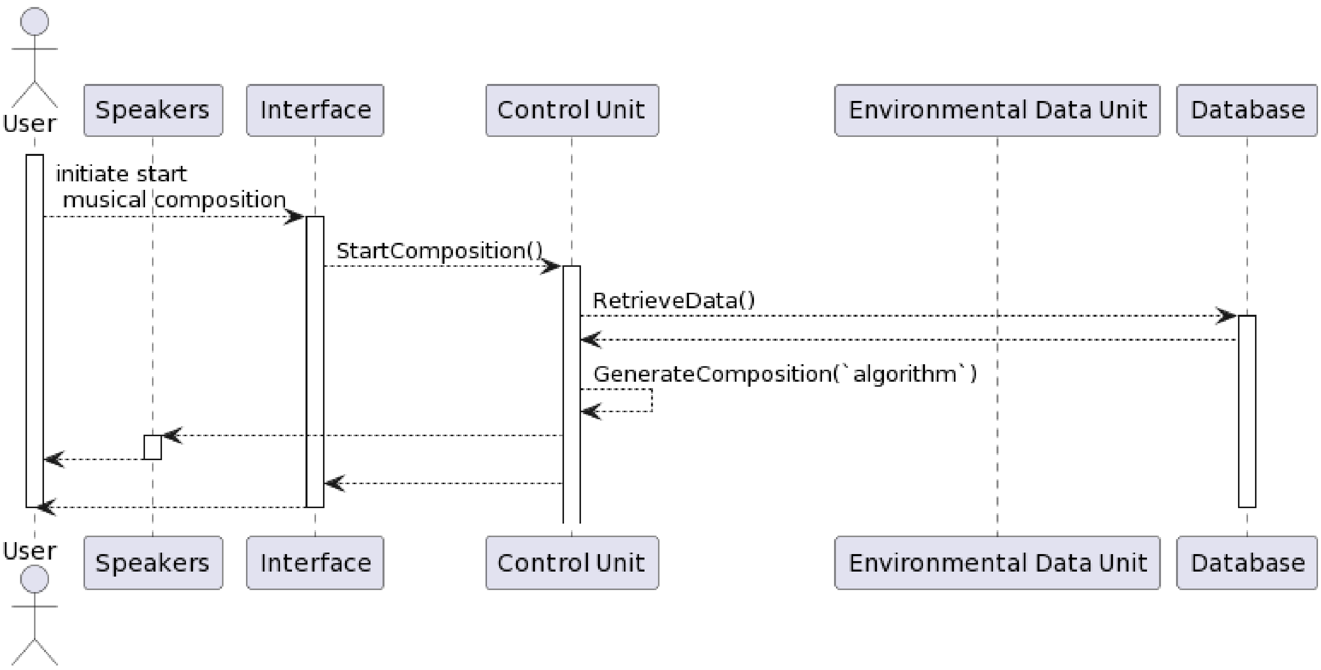


Figure 5. UC_02 Scenario 1

Scenario 2

When the user initiates the start of outputting musical compoition while no data has/is been captured, an error message occurs.

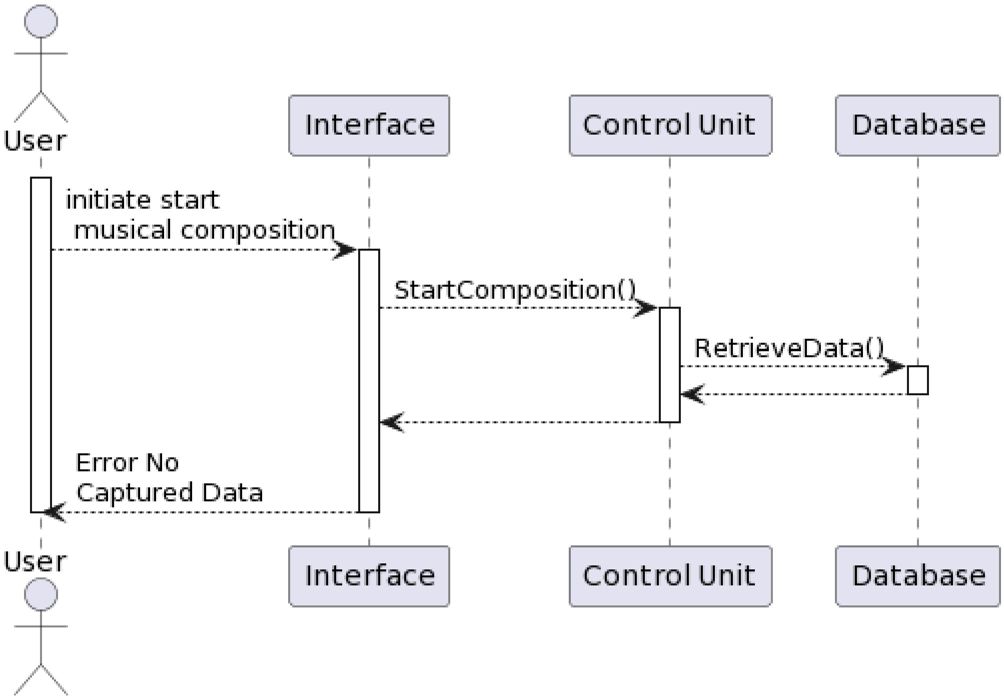


Figure 6. UC_02 Scenario 2

5.1.3. UC_03 : Change musical composition

Scenario 1

When the user initiates change in algoritm for the musical composition, the control unit switches the algoritm and generates the new musical composition.

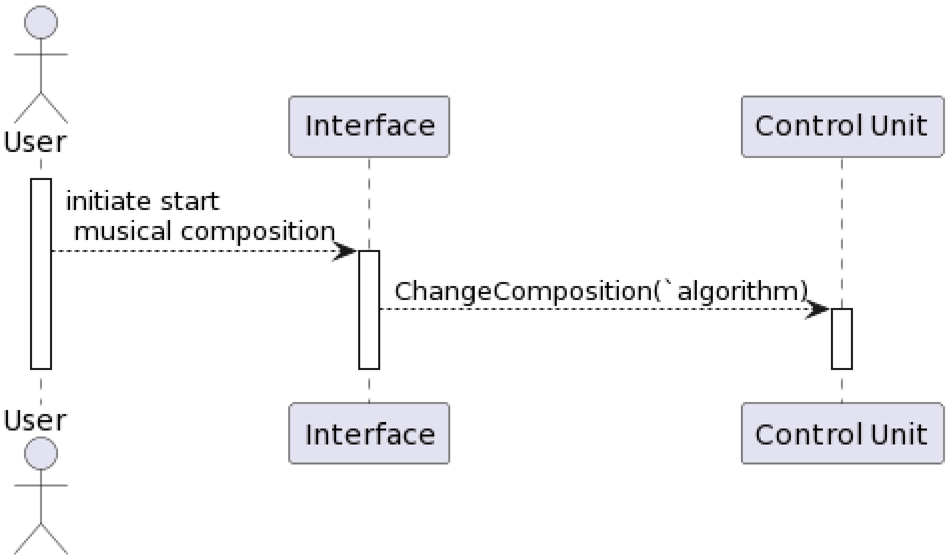
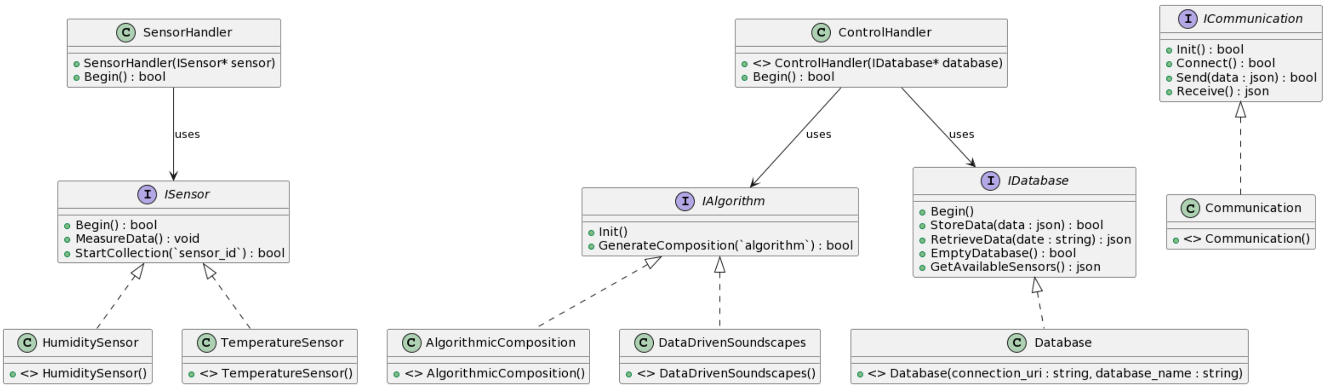


Figure 7. UC_03 Scenario 1

5.2. Class diagrams

A UML class diagram showcases how each interface/class operates on their own and in relation to interfaces and/or classes.

Within the class diagram, there is made use of various design pattern. For example, there has been made use of dependency injection. Dependency injection allows for possible extension of the system, as they are not hard-coded. Another design pattern that is used is the Strategy Pattern, as there is made use of various algorithms during runtime, and therefor they need to be interchangeable.



6. Hardware Diagram

The system will be composed of several hardware modules. Some logical units run on multiple physical blocks. The table, [Used hardware components], describes the individual hardware hardware components. The diagram, [Hardware connection], the connection between the various hardware components.

Table 12. Used hardware components

Hardware block	Description
Raspberry Pi 4	This device is responsible for making any type of decisions and overall generating the musical compositions.
Arduino Nano 33 IoT	This device is responsible for collecting the environmental data.
Environmental Sensors	There are various sensors used to measure data with the help of the Arduino Nano 33 IoT.
Speakers	This device is reposnible to allow for the generated musical composition to be heard.

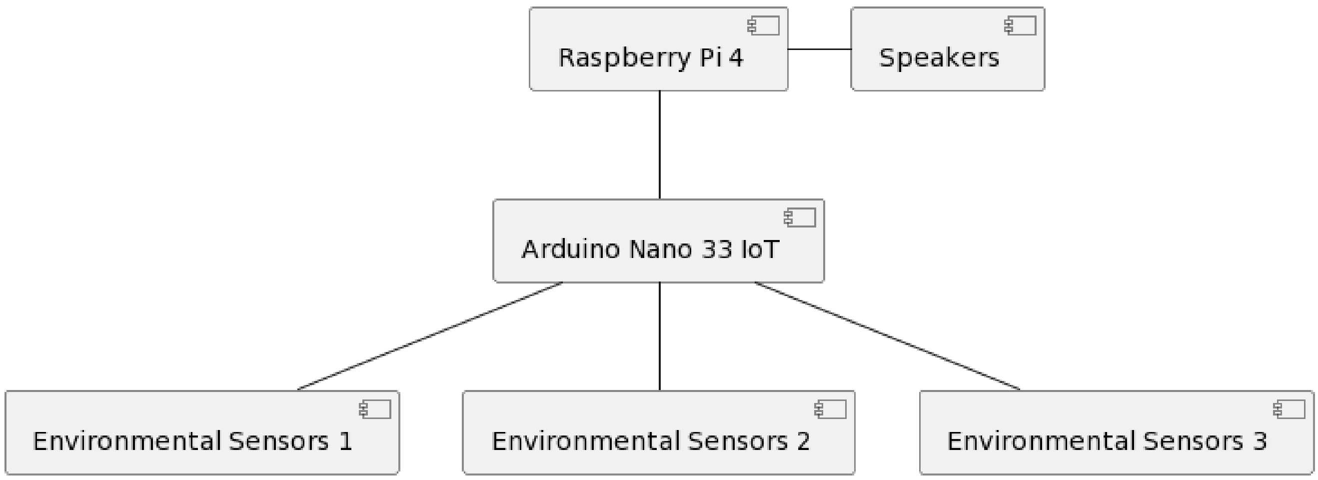


Figure 8. Hardware connection